hexens x KyberSwap

Security Review Report for KyberSwap

October 2026

Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - Recipient balance changes can bypass minPrice when dstFeeRate == 0
    - Zero-liquidity zap-outs can steal accrued fees
    - Leaf index naming obscures its role as a counter identifier
    - Native-token aliasing can pay V4 liquidity-removal proceeds twice
    - Uniswap V3 zap decodes return-data offsets incorrectly

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for Kyber Network's KyberSwap protocol. This scope included the introduction of a conditional swap hook.

Our security assessment was a full review of the code, spanning a total of 3 days.

During our review, we did not identify any major severity vulnerability.

We did identify some minor severity vulnerabilities and code optimizations.

All reported issues were fixed by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

▪ Review Led by

Kasper Zwijsen, Head of Audits

▪ Scope

The analyzed resources are located on:

🔗 https://github.com/KyberNetwork/smart-intent-sc/tree/0c5f825473c86d4875dd25945d388baadcd0c072

The issues described in this report were fixed in the following commit:

🔗 https://github.com/KyberNetwork/smart-intent-sc/tree/d4b773228430fdb19e81e6c554b864d687aa8705

▪ Changelog

12 October 2026	Audit start
19 October 2026	Initial report
22 October 2026	Revision received
30 October 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

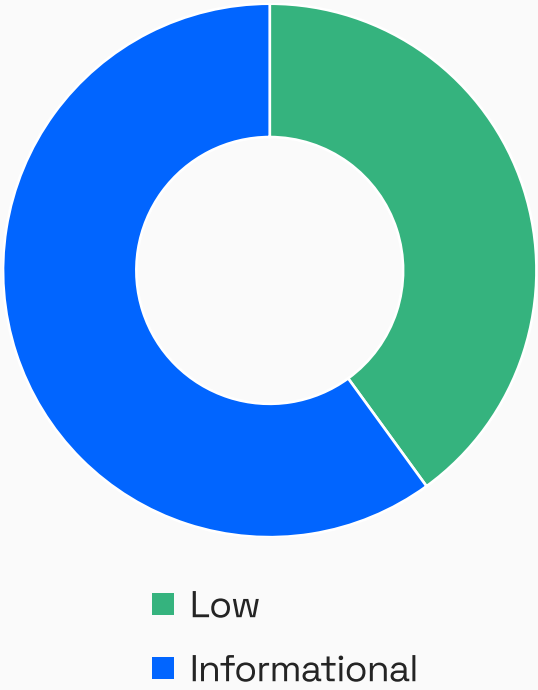
- **Issue Symbolic Codes**

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	0
Low	2
Informational	3
Total:	**5**





6. Weaknesses

This section contains the list of discovered weaknesses.

KYBER3-3 | Recipient balance changes can bypass minPrice when dstFeeRate == 0

Fixed

Severity:	Probability:	Impact:
Low	Unlikely	Low

Path:

./src/hooks/swap/KSConditionalSwapHook.sol

Description:

When **dstFeeRate == 0**, the hook measures the change in the recipient's token-out balance to determine whether the swap satisfies **minPrice**. However, this includes any tokens received during the action, not just the swap output.

According to the documented trust model, the delegated key is untrusted and can choose the swap route and fee rates within the limits committed to the Merkle leaf. A malicious delegated key can therefore execute a swap that pays less than the user's minimum price and use an external payout to make the hook accept it.

Root cause

In **beforeExecution**, the hook snapshots the balance of the recipient when **dstFeeRate == 0**.

```
holderBalanceBefore: tokenOut.balanceOf(
    _settlementHolder(swapHookData.recipient, intentDstFeeRate)
 ),

function _settlementHolder(address recipient, uint256 feeRate) internal view returns (address) {
  return feeRate != 0 ? msg.sender : recipient;
}
```

In **afterExecution**, it calculates the output from the difference between the recipient's current balance and that snapshot:

```
uint256 amountOut = tokenOut.balanceOf(
    _settlementHolder(validationData.recipient, validationData.dstFeeRate)
) - validationData.holderBalanceBefore;

uint256 netAmountOut = amountOut - dstFee;
uint256 netExecutionPrice = Math.mulDiv(
    netAmountOut, OracleLib.PRICE_SCALE, amountIn
);
```

The hook then checks **netExecutionPrice** against **minPrice**. Because the measurement covers the entire action, tokens received from other sources can satisfy the price check.

The documented invariant that the realized price is based on the actual swap output is therefore not enforced in this settlement mode.

Example

Alice signs an intent to sell 1 ETH for at least $2,970. Her employer also pays her salary through a Sablier stream, from which approximately $2,900 is available to withdraw.

The delegated key chooses a route that transfers Alice's ETH to the attacker, pays Alice $100 in USDC, and calls **Sablier.withdrawMax(streamId, alice)** to withdraw her accrued salary.

The hook observes approximately $3,000 in additional USDC and accepts the swap, even though the swap itself paid only $100. Alice loses her 1 ETH for $100, and the withdrawn salary makes the execution appear to satisfy her minimum price.

Remediation:

Always measure swap output at the router, regardless of **dstFeeRate**. Have the hook return the output tokens, fees and amounts to the router so it can transfer the net output to the committed recipient.

This ensures that only tokens received through the swap settlement count toward **minPrice**, preventing unrelated transfers to the recipient from satisfying the price check.

KYBER3-5 | Zero-liquidity zap-outs can steal accrued fees

Fixed

Severity:	Probability:	Impact:
Low	Unlikely	Low

Path:

kyberswap-oct-26/src/hooks/zap-out/KSZapOutUniswapV3Hook.sol:afterExecution#L102-L149
kyberswap-oct-26/src/hooks/zap-out/KSZapOutUniswapV4Hook.sol:afterExecution#L105-L149

Description:

The **afterExecution** hooks validate zap-outs from Uniswap V3 and V4 positions. After a whitelisted executor handles the position, each hook checks that the NFT was returned if liquidity remains and that the recipient received enough output for the liquidity removed.

A valid zap-out should only count proceeds from removed liquidity toward the minimum rate. Any fees already earned by the position belong to the user and should be returned separately, even when no liquidity is removed.

However, the hooks do not require any liquidity to be removed and do not account for accrued fees. When **liquidityBefore == liquidityAfter**, the minimum-output check becomes **outputAmount >= 0** and always passes:

```
liquidity = liquidityBefore - liquidityAfter;
if (outputAmount * RATE_DENOMINATOR < minRate * liquidity) revert BelowMinRate(...);
```

Collecting a V3 position, or decreasing a V4 position by zero, can still realize all accrued fees. A whitelisted executor can therefore collect those fees to an address chosen in the zap call, return the unchanged NFT, and pass every hook check. The same missing fee accounting also lets accrued fees cover part of the required output during a partial exit.

An exploit can happen as follows:

1. A user's V3 or V4 position accrues fees and becomes eligible for execution.
2. A whitelisted executor submits a zap-out that removes zero liquidity and sends the collected fees to its chosen recipient.
3. The position NFT is returned to the user with all liquidity intact.
4. Since the removed liquidity and required output are both zero, **afterExecution** accepts the execution while the executor keeps the fees.

Remediation:

Require **liquidityBefore - liquidityAfter** to be greater than zero. Also calculate the position's accrued fees on-chain and require the recipient to receive those fees in addition to the minimum output for the removed liquidity.

KYBER3-1 | Leaf index naming obscures its role as a counter identifier

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

src/hooks/swap/KSConditionalSwapHook.sol

Description:

KSConditionalSwapHook.getSwapExecutionCount() uses **leafIndex** to select an execution counter.

```
function getSwapExecutionCount(bytes32 intentHash, uint256 leafIndex)
  public
  view
  returns (uint256)
{
  uint256 packedValue = swapRecord[intentHash][leafIndex / 32];
  uint256 bytePosition = leafIndex % 32;
  ...
}
```

The name **leafIndex** suggests a position in the Merkle tree, but it represents a logical counter identifier committed within the leaf hash. Its numerical value does not determine tree position: a leaf with identifier **0** may appear to the right of a leaf with identifier **1**.

An integration that interprets this identifier as a tree position may submit the position instead of the committed identifier. When these values differ, **beforeExecution()** computes a different leaf hash and reverts with **InvalidProof**, preventing the affected execution.

Remediation:

- Rename **leafIndex** to **conditionId** consistently across leaf hashing, action decoding, callback data, and counter access.
- Represent committed identifiers and tree positions separately in leaf construction and action encoding code.

KYBER3-2 | Native-token aliasing can pay V4 liquidity-removal proceeds twice

Fixed

Severity:	Probability:	Impact:
Informational	Unlikely	Informational

Path:

src/hooks/remove-liq/KSRemoveLiquidityUniswapV4Hook.sol:_cacheValidationData#L66-L93
src/hooks/remove-liq/KSRemoveLiquidityPancakeV4CLHook.sol:_cacheValidationData#L67-L99

Description:

These functions prepare output validation for Uniswap V4 and PancakeSwap V4 CL liquidity removals. They read the two pool currencies, normalize native-token addresses, and record the router's balances before the position manager removes liquidity. After removal, **BaseTickBasedRemoveLiquidityHook._validateOutput** calculates each output from the balance increase, and **HookLibrary.afterExecution** transfers both outputs to the position owner.

Normally, the two output entries represent two different assets. Each balance increase should therefore be counted once and only the assets received from the removal should be paid to the user.

However, a pool can use **address(0)** as its native currency and **TokenHelper.NATIVE_ADDRESS** as its other currency. V4 considers these different addresses, but the hooks normalize both to **TokenHelper.NATIVE_ADDRESS**. If wrapping is enabled, both are then normalized to WETH instead. The hook does not check that the resulting output tokens are still different.

As a result, **_recordRouterBalances** reads the same asset balance twice. A single increase of **X** is reported as two increases of **X**, and **HookLibrary.afterExecution** transfers both amounts. The second transfer comes from native tokens or WETH already held by the router.

An attacker can exploit this as follows:

1. Create a V4 pool using **address(0)** and **TokenHelper.NATIVE_ADDRESS**, then mint a one-sided position funded only with native currency.
2. Submit a valid removal intent while the router already holds native tokens or WETH, depending on the wrapping setting.
3. Remove **X** from the position. Both output slots record the same **X** balance increase.
4. The settlement loop sends X twice, returning the attacker's principal and taking another X from the router's existing balance.

Only pre-existing balance in the router is affected.

```
function _recordRouterBalances(address router, address[2] memory tokens)
  internal
  view
  returns (uint256 balance0, uint256 balance1)
{
  balance0 = tokens[0].balanceOf(router);
  balance1 = tokens[1].balanceOf(router);
}
```

Remediation:

After all native-token and WETH normalization, require the two output token addresses to be different in both V4 hooks. As defense in depth, settlement should also reject or combine duplicate output-token entries before transferring funds.

KYBER3-4 | Uniswap V3 zap decodes return-data offsets incorrectly

Fixed

Severity:	Probability:	Impact:
Informational	Unlikely	Informational

Path:

src/hooks/zap-out/KSZapOutUniswapV3Hook.sol:_getPositionLiquidity,_getSqrtPriceX96#L152-L175

Description:

The Uniswap V3 zap hook checks the pool price before execution and measures how much liquidity was removed afterward. These values are used to keep execution within the user's signed price range and minimum output rate.

The configured offsets are ABI return-word indexes. The readers should skip the 32-byte **bytes** length field and then move by 32 bytes for each word. For example, offset **7** should read word 7 from **positions()**, while offset 0 should read the first word from **slot0()**.

However, both readers add the configured index directly to the **bytes** pointer:

```
liquidity := mload(add(result, liquidityOffset))
sqrtPriceX96 := mload(add(result, priceOffset))
```

This treats a word index as a raw byte offset and does not skip the length field. With the offsets used by the repository test, **slot0()** returns the constant return-data length **224** instead of the pool price. The liquidity reads also return the same header-derived value before and after execution, making the removed liquidity equal to zero. The minimum-rate check then always passes because it compares the output against **minRate * 0**.

An example flow is:

1. A user signs a zap intent with liquidity offset **7**, price offset **0**, and a broad price range that includes **224**.
2. The hook accepts the fake price and records a header-derived liquidity value before execution.
3. The zap removes the position's liquidity and executes with an action-level output minimum below the intent's minimum rate, for example after a stale quote or sandwich.
4. The hook reads the same fake liquidity value afterward, calculates zero removed liquidity, and accepts the lower output.

The separate action-level minimum still limits the final loss. A narrow, realistic pool-price range instead causes the zap to revert every time.

Remediation:

Use typed Uniswap V3 interface calls and destructure the required values. If configurable word indexes are needed, validate the return-data length and load each value from **result + 0x20 + wordIndex * 0x20**.


hexens
x
KyberSwap